



Skills You Need to Learn for v1

1. Backend Basics with Supabase (5–6 hours learning)

- Setting up Supabase project and database tables
- Writing simple SQL queries for CRUD operations
- Understanding Row Level Security (RLS) for user data isolation
- Supabase Auth for login/signup
- Supabase Storage for PDF file uploads

Resources:

- Supabase official docs: <https://supabase.com/docs>
- YouTube: "Supabase crash course for React developers" (~2 hours)

2. Email Webhook Integration (3–4 hours learning)

- How email webhooks work (SendGrid Inbound Parse)
- Parsing incoming webhook payloads (JSON structure)
- Extracting email attachments from base64 encoded data
- Setting up MX records in your domain DNS

Resources:

- SendGrid Inbound Parse documentation
- Tutorial: "Receive emails in Node.js app" (1 hour read)

3. OCR/PDF Parsing (5–6 hours learning)

- Google Vision API basics for document text extraction
- Understanding OCR response structure (text blocks, confidence scores)
- Regex patterns for extracting invoice fields (amount, date, invoice number)
- Handling PDF to image conversion (if needed)

Resources:

- Google Vision API quickstart for Node.js
- Invoice parsing examples on GitHub (search "invoice OCR node")

4. Basic Node.js/Express API (4–5 hours learning)

- Creating REST API endpoints (POST, GET, PUT)
- Handling file uploads and parsing
- Environment variables for API keys
- Deploying serverless functions (Vercel/Supabase Edge Functions)

Resources:

- "Node.js Express tutorial for beginners" (~3 hours)
- Vercel serverless functions docs (1 hour)

5. Database Schema Design (2–3 hours learning)

- Designing normalized tables for invoices, users, vendors
- Understanding foreign keys and relationships
- Basic indexing for query performance

Resources:

- "Database design for beginners" YouTube video
- [Draw.io](https://draw.io) or dbdiagram.io for visualizing schema

Total Learning Time: 20–25 hours (can be done in 2 weeks of evenings)

v1 Build Strategy

Phase 1: Foundation (Week 1)

Goal: Get authentication and basic dashboard working

1. **Set up Supabase project** (1 hour)
 - Create account, new project
 - Note down API keys and URL
2. **Design database schema** (2 hours)
 - Tables: users, invoices, vendors
 - Write SQL in Supabase SQL editor
3. **Build React app skeleton** (3 hours)
 - Create React app with Vite
 - Set up routing (React Router)
 - Add Tailwind CSS or your preferred UI library
4. **Implement authentication** (4 hours)

- Supabase Auth signup/login forms
- Protected routes
- User session management

5. **Create empty dashboard layout** (2 hours)

- Header with user email and logout
- Sidebar or top nav
- Empty invoice list placeholder

Week 1 total: 12 hours

Output: You can sign up, log in, see empty dashboard

Phase 2: Core Invoice Features (Week 2)

Goal: Manual invoice entry and display working

1. **Invoice list table UI** (4 hours)

- React table component with columns: vendor, number, date, amount, status
- Filters (status dropdown, date range)
- Search bar (vendor or invoice number)

2. **Add invoice manually form** (3 hours)

- Modal/drawer with form fields
- Form validation
- Submit → save to Supabase

3. **Invoice detail view** (2 hours)

- Click row → side panel with full details
- Edit invoice fields
- Delete invoice

4. **Fetch and display real data** (3 hours)

- Supabase queries to load invoices
- Real-time updates (optional but cool with Supabase)
- Loading states and error handling

Week 2 total: 12 hours

Output: You can manually add, view, edit, delete invoices

Phase 3: Email Integration (Week 3)

Goal: Email forwarding works, invoices auto-added

1. Set up domain and SendGrid (2 hours)

- Buy domain or use existing
- Configure MX records
- Set up SendGrid Inbound Parse webhook

2. Create webhook endpoint (4 hours)

- Node.js/Express endpoint or Supabase Edge Function
- Receive POST from SendGrid
- Parse email to field to identify user
- Extract PDF attachments

3. Save PDFs to Supabase Storage (2 hours)

- Upload received PDFs
- Generate public URL
- Link to invoice record

4. Show unique email in dashboard (2 hours)

- Display `invoices+username@yourapp.com` in UI
- Copy to clipboard button
- Simple onboarding instructions

Week 3 total: 10 hours

Output: Users can forward emails, PDFs get saved, visible in dashboard (but no OCR yet)

Phase 4: OCR and Auto-Extraction (Week 4)

Goal: PDFs automatically parsed into invoice fields

1. Set up Google Vision API (1 hour)

- Create Google Cloud project
- Enable Vision API
- Get API credentials

2. Implement OCR parsing (5 hours)

- Call Vision API with PDF/image
- Extract raw text
- Write regex/rules to find: vendor name, invoice number, amount, date, due date

3. Auto-populate invoice fields (3 hours)

- When email arrives → OCR → create invoice record with extracted data

- Flag "needs review" if confidence is low
- Allow user to edit/fix fields

4. **Send confirmation email** (2 hours)

- After processing, email user: "Invoice #123 added!"
- Include link to view in dashboard

Week 4 total: 11 hours

Output: Full workflow—forward email → auto-extracted → appears in dashboard

Phase 5: Payment Status and Reports (Week 5)

Goal: Track payment status, show basic reports

1. **Payment status functionality** (3 hours)

- Three statuses: Unpaid, Paid, Overdue
- Auto-calculate Overdue based on due date
- Manual "Mark as Paid" button with date picker

2. **Basic dashboard reports** (3 hours)

- Total outstanding amount
- Total overdue amount
- Paid this month
- Simple cards/widgets at top of dashboard

3. **Vendor directory** (2 hours)

- Auto-populate vendors from invoices
- List view with total outstanding per vendor
- Click vendor → filter invoices by vendor

Week 5 total: 8 hours

Output: Full v1 feature set complete

Phase 6: Testing, Polish, Deploy (Week 6)

Goal: Production-ready app with beta users

1. **Bug fixes and edge cases** (4 hours)

- Test with different invoice formats
- Handle missing fields gracefully
- Fix OCR parsing errors

2. **UI polish** (3 hours)

- Clean up styles

- Add loading spinners
- Better error messages
- Mobile responsiveness check

3. Deploy to production (2 hours)

- Deploy frontend to Vercel
- Set up production Supabase project
- Configure production domain and email

4. Onboard 3–5 beta users (3 hours)

- Create accounts for them
- Give them instructions
- Watch them use it (screenshare or in-person)

Week 6 total: 12 hours

Output: v1 live in production with real users

Time Commitment Summary

Week	Phase	Hours	Cumulative
Week 1	Foundation	12	12
Week 2	Core Invoice Features	12	24
Week 3	Email Integration	10	34
Week 4	OCR Auto-Extraction	11	45
Week 5	Payment Status & Reports	8	53
Week 6	Testing, Polish, Deploy	12	65 hours

Realistic Timeline with 9-5 Job:

If you commit 15 hours/week: $65 \text{ hours} \div 15 = 4.3 \text{ weeks}$ → Launch by **mid-March 2026**

If you commit 20 hours/week: $65 \text{ hours} \div 20 = 3.2 \text{ weeks}$ → Launch by **early March 2026**

If life happens and you do 10 hours/week: $65 \text{ hours} \div 10 = 6.5 \text{ weeks}$ → Launch by **end of March 2026**

Summary: Key Pointers from Entire Chat

1. The Problem You're Solving

- Indian micro-SMEs waste 10+ hours/week on invoice management^{[1] [2]}
- Invoices scattered across email, WhatsApp, paper; payments hard to track^{[3] [4]}
- Manual reconciliation, missed follow-ups, GST compliance stress^{[2] [5]}
- Existing tools too complex or don't fit their WhatsApp-first workflow^{[6] [1]}

2. Your Solution (v1 Scope)

- **One inbox for all invoices:** Email forwarding → auto-capture → dashboard^{[7] [8]}
- **Auto-extraction via OCR:** PDFs parsed, fields populated automatically^{[9] [10]}
- **Payment status tracking:** Unpaid/Paid/Overdue with manual updates^{[11] [7]}
- **Basic reports:** Outstanding, overdue, paid amounts^{[12] [11]}
- **NOT a super app:** Focused on invoice workflow, not full accounting^{[13] [6]}

3. Why This Fits You Specifically

- Matches your React/frontend strengths perfectly
- Backend learning is manageable with modern tools (Supabase)^[14]
- You've built admin dashboards before—this is similar^{[15] [16]}
- Low financial risk (₹0–₹3K for v1) addresses your insecurities^[14]
- Can build while keeping 9-5 job = safe experimentation

4. Tech Stack (Keep It Simple)

- **Frontend:** React + Tailwind CSS (your comfort zone)^[16]
- **Backend/DB:** Supabase (auth, database, storage—all in one)^[14]
- **Email:** SendGrid Inbound Parse (free tier: 100 emails/day)^[8]
- **OCR:** Google Vision API (free tier: 1,000 pages/month)^[9]
- **Hosting:** Vercel (frontend free), Supabase (backend free tier)^[14]

5. Initial Costs

- **v1 (email-only):** ₹0–₹1,200/year (just domain, optional)^{[9] [14]}
- **v2 (add WhatsApp):** ₹1,500–₹3,000/month for 50 users^{[17] [18]}
- **Scale (100 users):** ₹6,500–₹9,500/month^{[9] [14]}
- **Your pricing:** ₹400–₹600/user/month = profitable from day 1^{[6] [13]}

6. Three-Launch Roadmap

- **v1 (Weeks 1–6):** Email forwarding + OCR + dashboard + manual payment tracking
- **v2 (Weeks 7–10):** Add WhatsApp forwarding + payment matching + email reminders^[19] ^[3]
- **v3 (Weeks 11–14):** Auto reminders + GST exports + payment links + team mode^[5] ^[17]

7. The "Share to Us" Flow (Key Innovation)

- Users forward invoices from personal email/WhatsApp to your unique address^[3] ^[8]
- **Zero friction:** Takes 10 seconds vs. 2–3 minutes manual upload^[4] ^[20]
- **Mobile-first:** Works perfectly on phone where 90% of SMEs operate^[21] ^[22]
- **No app switching:** Uses tools they already have open 24/7^[3]

8. Can You Build This Solo?

- **Yes:** 70% is React (your strength), 30% is learnable backend basics^[16]
- **Time needed:** 65 hours for v1 = 4–6 weeks at 10–15 hours/week with day job
- **Learning needed:** 20–25 hours total (backend, email webhooks, OCR)
- **Many solo founders:** Have built similar B2B SaaS while working full-time^[13]

9. Validation Milestones

- **Month 1–2:** Build v1, get 5–10 beta users from Surat network
- **Month 3:** If 3+ say "I'll pay ₹300–₹500/month" → build v2
- **Month 4–6:** Get 20–30 paying users, achieve ₹6K–₹15K MRR
- **Decision point:** 50+ users or ₹20K+ MRR = consider full-time, else keep as side income

10. Managing Insecurities

- **Start small:** v1 is just a tool, not "I'm a startup founder now"
- **Keep job:** No financial pressure, experiment freely
- **Low cost:** ₹15K total over 6 months vs. ₹5–10L for traditional business
- **Skill insurance:** Even if it fails, you're a better engineer with backend + SaaS experience
- **Portfolio boost:** Real product > demo projects for future opportunities^[23]

11. Success Definition

- **Short-term:** 10 active users forwarding invoices regularly
- **Medium-term:** 3–5 paying customers at ₹400+/month
- **Long-term:** 100 users, ₹50K MRR, NPS > 40 (strong customer love)^[24]
- **Alternative win:** ₹3–5L/month side income indefinitely without quitting job

12. Next Immediate Steps

1. **This weekend:** Set up Supabase account, create React app skeleton (3 hours)
2. **Next week:** Build auth + basic dashboard layout (12 hours)
3. **Week 3:** Get email forwarding working (10 hours)
4. **Week 4:** Add OCR (11 hours)
5. **Week 5:** Payment tracking (8 hours)
6. **Week 6:** Polish + deploy + get first beta user (12 hours)

Final Motivation

You have:

- ✓ The right skills (React, frontend)
- ✓ The right mindset (want to learn, build business)
- ✓ Access to target market (Surat has thousands of micro-SMEs)
- ✓ Low-risk opportunity (minimal investment)
- ✓ Clear validation path (beta → paid → scale)

What you need:

- ☐ Commitment to 15 hours/week for 6 weeks
- ☐ Permission to build v1 ugly (users care about pain solved, not polish)
- ☐ Courage to talk to 10 real SME owners (this is harder than coding)

The business knowledge you want will come from building this, not from reading about it.

Start this weekend.

✱

1. <https://www.saniiro.com/article/billing-and-inventory-challenges-for-indian-businesses>
2. <https://www.cnbctv18.com/business/msmes-fear-annual-burden-as-india-rolls-out-new-gst-invoice-management-system-19580642.htm>
3. <https://www.billinsecond.com/post/best-practices-whatsapp-invoice-send>
4. <https://www.insidash.com/blogs/share-invoices-via-whatsapp-billing-software>
5. <https://gsthero.com/blog/benefits-of-e-invoicing-for-msmes-advantages-need/>
6. <https://hisabkitab.co/top-10-billing-and-invoicing-software-for-smes>
7. <https://razorpay.com/x/vendor-payments/>
8. <https://zapier.com/templates/details/07be9e10a4ae32192bed0b28b0fe7db79cdeb60514b2ef32fd3af35d39a36cfb>
9. <https://programminghistorian.org/en/lessons/ocr-with-google-vision-and-tesseract>
10. <https://www.eklavya.com/blog/best-ocr-answersheet-evaluation/>
11. <https://collabcrm.com/blog/best-invoice-management-software/>

12. <https://spenmo.com/blog/what-is-invoice-management>
13. <https://www.smarket.ai/blog/invoicing-software-for-indian-smes-a-complete-guide/>
14. skills.frontend_react
15. projects.resume_website_stack
16. <https://dev.to/wahee/supabase-vs-firebase-free-tier-2025-which-one-should-you-use-5cmp>
17. <https://www.perplexity.ai/search/ba5516dd-b0ca-4330-bc2b-450a04f6038b>
18. <https://www.perplexity.ai/search/c852a64e-cb68-4d1e-ac8f-5a13e03c9591>
19. bio.age_and_role
20. <https://www.a2zsms.in/blogs/whatsApp-business-aPI-pricing-in-india-everything-you-need-to-know/>
21. <https://www.wassy.in/whatsapp-api-price-comparison>
22. <https://whapi.cloud>
23. <https://msgbill.com>
24. <https://quantiqueminds.com/insights/whatsapp-commerce-revolution.html>
25. <https://kinaracapital.com/whatsapp-business-empowering-msmes/>
26. <https://www.perplexity.ai/search/dc0275e1-b436-4203-8008-2d0c10da2913>
27. <https://contentsquare.com/guides/net-promoter-score/>